

Spike: 1**Title:** GridWorld

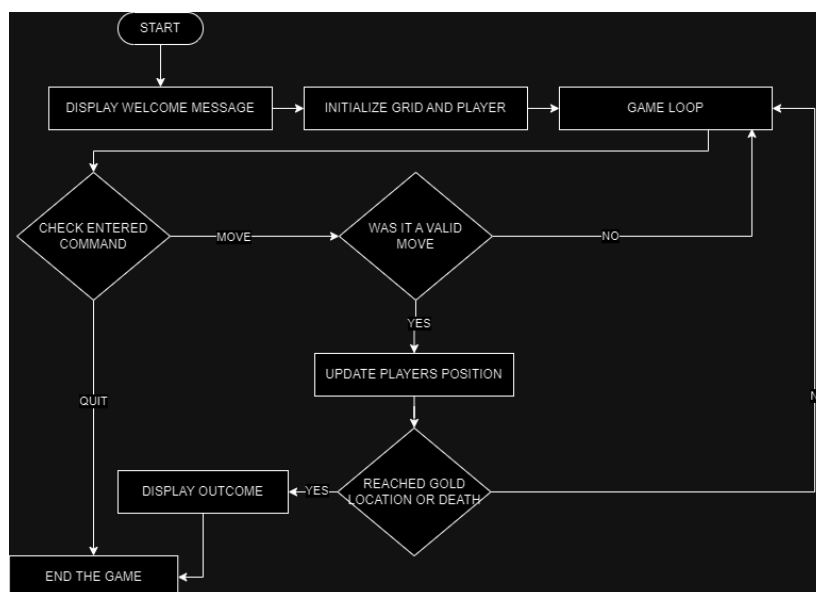
Author: Shafi Uzman Fassy (102608927)

Goals / deliverables:

Creating the gridworld game as specified in the specifications document using C++ coding language and demonstrate the use of simple game loop, separation of update / render code and the use of game data.

Besides this report, we created C++ code and a flow chart as shown below

- For code see COS30031-2023-102608927\03 - Spike - Gridworld
- Simple Planning Flow Chart

**Technologies, Tools, and Resources used:**

List of information needed by someone trying to reproduce this work

- Visual Studio Code
- MinGW
- C++ Cheat Sheet <https://www.codewithharry.com/blogpost/cpp-cheatsheet/>

Tasks undertaken:

To allow another coder to reproduce this work, they would need to follow these steps.

- Download and install Visual Studio Code
- Download and install MinGW.
- Configure Path to MinGW directory.
- Create a simple plan laying out what is required to achieve making GridWorld.
- Creating a new C++ file in visual studio code.
- Declaring variables to create the grid.

```
const char Grid_Size = 8;  
  
const char EMPTY = ' ';  
const char WALL = '#';  
const char START = 'S';  
const char GOLD = 'G';  
const char DEATH = 'D';
```

- Create a structure to store player's position.

```
struct PlayerPosition  
{  
    char x;  
    char y;  
};
```

- Create a function to create and display the grid layout.

```
void displayGrid(const char grid[Grid_Size][Grid_Size], const PlayerPosition& playerPosition) {  
    for (int x = 0; x < Grid_Size; ++x) {  
        for (int y = 0; y < Grid_Size; ++y) {  
            if (x == playerPosition.x && y == playerPosition.y) {  
                cout << 'P'; // Display the player's position  
            } else {  
                cout << grid[x][y];  
            }  
        }  
        cout << endl;  
    }  
}
```

- Declaring bool variables for game conditions.

```
bool isGameOver = false;  
bool isPlayerWon = false;  
char user_cmd;
```

- Implement a switch case game loop to update players positions based on the user command entered that moves players direction.

```
while (!isGameOver)
{
    displayGrid(grid, playerPosition);

    cout << "Enter Your Command: ";
    cin >> user_cmd;

    user_cmd = toupper(user_cmd);

    switch (user_cmd)
    {
    case 'N':
        if (playerPosition.x > 0 && grid[playerPosition.x - 1][playerPosition.y] != WALL)
        {
            --playerPosition.x;
        }
        cout << "You have chosen to go North" << endl;
        break;
    case 'S':
        if (playerPosition.x < Grid_Size - 1 && grid[playerPosition.x + 1][playerPosition.y] != WALL)
        {
            ++playerPosition.x;
        }
        cout << "You have chosen to go South" << endl;
        break;
    case 'E':
        if (playerPosition.y < Grid_Size - 1 && grid[playerPosition.x][playerPosition.y + 1] != WALL)
        {
            ++playerPosition.y;
        }
        cout << "You have chosen to go East" << endl;
        break;
    case 'W':
        if (playerPosition.y > 0 && grid[playerPosition.x][playerPosition.y - 1] != WALL)
        {
            --playerPosition.y;
        }
        cout << "You have chosen to go West" << endl;
        break;
    case 'Q':
        cout << "You have chosen to quit the game" << endl;
        isGameOver = true;
        break;
    }
}
```

- Implement game conditions based on if player found the goal or touched death.

```
You, 6 days ago • implemented game conditions ~
    if (grid[playerPosition.x][playerPosition.y] == GOLD)
    {
        isPlayerWon = true;
        isGameOver = true;
    }
    else if (grid[playerPosition.x][playerPosition.y] == DEATH)
    {
        cout << "Wow - you've discovered a large chest filled with GOLD coins!" << endl;
        cout << "YOU WIN!" << endl;
        cout << "Arrrrgh... you've fallen down a pit." << endl;
        cout << "YOU HAVE DIED!" << endl;
        isPlayerWon = false;
        isGameOver = true;
    }
}
```

- Finally implementing a win message if player won.

```
if (isPlayerWon)
{
    cout << "Thanks for playing. Maybe next time." << endl;
} else {
    cout << "Thanks for playing. There probably won't be a next time." << endl;
}
```

What we found out:

We found out that the player was able to move / teleport past walls so to fix this issue we had to update our switch case with else conditions to prevent player from moving towards a wall from that would be in the players way.

```
switch (user_cmd)
{
    case 'N':
        if (playerPosition.x > 0 && grid[playerPosition.x - 1][playerPosition.y] != WALL)
        {
            --playerPosition.x;
            cout << "You have chosen to go North" << endl;
        }
        else
        {
            cout << "You cannot go that way!" << endl;
        }
        break;
    case 'S':
        if (playerPosition.x < Grid_Size - 1 && grid[playerPosition.x + 1][playerPosition.y] != WALL)
        {
            ++playerPosition.x;
            cout << "You have chosen to go South" << endl;
        }
        else
        {
            cout << "You cannot go that way!" << endl;
        }
        break;
    case 'E':
        if (playerPosition.y < Grid_Size - 1 && grid[playerPosition.x][playerPosition.y + 1] != WALL)
        {
            ++playerPosition.y;
            cout << "You have chosen to go East" << endl;
        }
        else
        {
            cout << "You cannot go that way!" << endl;
        }
        break;
    case 'W':
        if (playerPosition.y > 0 && grid[playerPosition.x][playerPosition.y - 1] != WALL)
        {
            --playerPosition.y;
            cout << "You have chosen to go West" << endl;
        }
        else
        {
            cout << "You cannot go that way!" << endl;
        }
        break;
    case 'Q':
        cout << "You have chosen to quit the game" << endl;
        isGameOver = true;
        break;
}
```